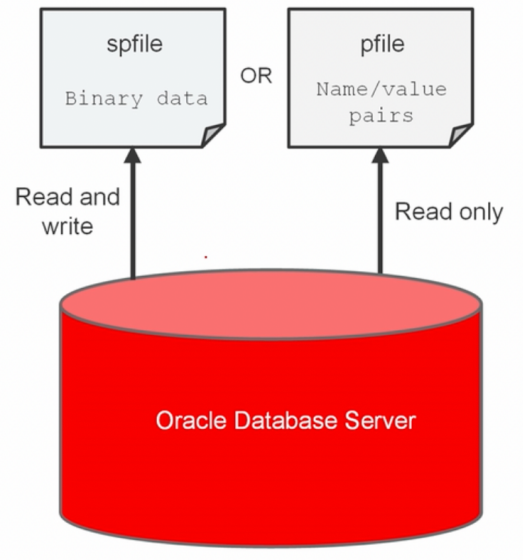
Managing Database Instances(Gestion de l'instance de la BDD)

Objectives

After completing this lesson, you should be able to:

- Describe initialization parameter files and initialization parameters.
- View and modify initialization parameters in SQL*Plus
- Work with the Automatic Diagnostic Repository (ADR)
- Query dynamic performance views (ex. The state of that running instance)

Working with Initialization Parameters



Pfile= initSID.ora
DBCA use init.ora

Initialization Parameters:

- Set database limits.
- Set database-wide defaults.
- Specify files and directories.
- Affect performance.

- Parameters can be of two types: basic or advanced.
  - Tune around 30 basic parameters to get reasonable database performance.
  - Example of an advanced parameter: DB_CACHE_SIZE
- Derived parameters calculate their values from the values of other parameters.
  - Example: SESSIONS is derived from PROCESSES.
- Some parameter values ranges depend on the host operating system.
- Some parameter values or value ranges depend on the host operating system.
  - Example: DB_BLOCK_SIZE

Modifying Initialization Parameters

- Modify parameters to set capacity limits or improve performance.
  - Use Enterprise Manager or SQL*Plus (**ALTER SESSION** or **ALTER SYSTEM**).
- Query **V$PARAMETER** for an initialization parameter to learn whether you can make :
  - Session-level changes (**ISSES_MODIFIABLE** column)
  - System-level changes (**ISSYS_MODIFIABLE** column)
  - PDB-level changes (**ISPDB_MODIFIABLE** column)
- Static parameter: that means cannot be change in running session running instance➔ shutdown the database and restart
- Use the SCOPE clause with the ALTER SYSTEM command to tell the system where to update the system-level parameter:
  - MEMORY (only on the running instance): dynamic
  - SPFILE static: in the file not in instance ➔ static
  - BOTH
- Use the DEFERRED keyword to set or modify the value of the parameter for future sessions that connect to the database.

Viewing Initialization parameters

Ways to view initialization parameters is SQL*Plus:

- Issue the SHOW PARAMETERS command.
  - Example: Find out about all the parameters whose names contain the word "para"

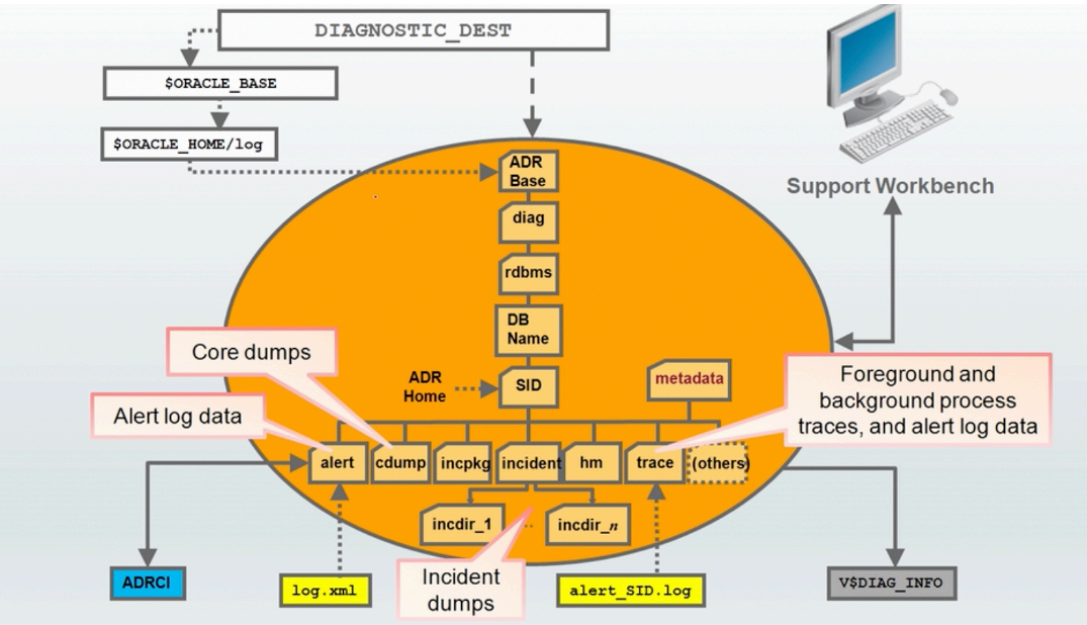
```
SQL> SHOW PARAMETER para
```

- Query the following views:
  - V$PARAMETER
  - V$PARAMETER2
  - V$SPPARAMETER
  - V$SYSTEM_PARAMETER
  - V$SYSTEM_PARAMETER2

The Automatic Diagnostic Repository (ADR):

- Is a file-based repository outside the database
- Is a system-wide central tracing and logging repository
- Stores database diagnostic data such as:
  - Traces
  - Alert log
  - Health monitor reports

Automatic Diagnostic Repository



Viewing the Alert Log

⇒ The alert log file is a chronological log of messages about the database instance and database, such as:

- Any nondefault initialization parameters used at startup.
- All internal errors (ORA-600), block corruption errors (ORA-1578), and deadlock errors(ORA-600) that occurred
- Administrative operations, such as the SQL statements CREATE, ALTER, DROP DATABASE, and TABLESPACE, and the Enterprise Manager or SQL*Plus statements STARTUP, SHUTDOWN, ARCHIVE LOG, and RECOVER
- Several messages and errors relating to the functions of shared server and dispatcher processes.
- Errors during the automatic refresh of a materialized view

⇒ Query $DIAG_INFO to find the location of the alert log.

- The path to alert_SID.log corresponds to the Diag trace entry
- The path to log.xml corresponds to the Diag Alert entry

⇒ You can view the alert log in a text editor or in ADRCI

Using Trace Files

⇒ Trace files contain:

- Error information (contact Oracle Support Services if an internal error occurs) is written to the corresponding trace file
- Information that can provide guidance for tuning applications or an instance

⇒ Each several and background process can write to an associated trace file

⇒ Trace file names for background processes are named after their processes.

- Exception: trace files generated by job queue processes

⇒ Oracle Database includes an advanced fault diagnosability infrastructure for preventing, detecting, diagnosing, and resolving problems.

⇒ When a critical error occurs:

- An incident number is assigned to the error
- Diagnostic data for the error (such as trace files) is immediately captured and tagged with the incident number

- Data is stored in the ADR

⇒ ADR files can be automatically purged by setting retention policy parameters

ADR (Automatic Diagnostic Repository):

- The ADR is a system wide tracing and logging central repository for database diagnostic data such as trace, the alert log, health monitor reports.
- The ADR root directory is known as ADR base. Its location is set by the DIAGNOSTIC_DEST initialization parameter.
- The location of ADR home is given as below:

```
Diag/product_type/product_id/db_id/instance_id
```

Administrating the DDL Log File (if you enable DDL_LOGING)

⇒ Enable the capture of certain DDL statements to a DDL log file by setting ENABLE_DDL_LOGGING to TRUE.

⇒ The DDL log contains one log record for each DDL statement.

⇒ Two DDL logs containing the same information:

- XML DDL log: log.xml written to $ORACLE_BASE/diag/rdbms/<dbname>/<SID>/log/ddl
- Text DDL: ddl_<sid>.log written to $ORACLE_BASE/diag/rdbms/<dbname>/<SID>/log

⇒ Example:

```
$ more ddl_orcl.log
Thu Nov 15 08:35:17 2016
diag_adl:drop user app_user
```

To be able to interact with these different things withing the Automatic diagnostic Repository we have ADRCI. Is command line tool that we can use to interact

Querying Dynamic Performance Views

⇒ Dynamic performance views provide access to information about the changing states of instance memory structures:

- Sessions, file states, and locks
- Progress of jobs and tasks
- Backup status, memory usage, and allocation
- System and session parameters
- SQL execution
- Statistics and metrics

⇒ Dynamic performance views start with the prefix V$

⇒ Example query: Which current sessions have logged in from the EDXX9P1 computer on the last day?

```
SQL> SELECT * FROM V$SESSION WHERE machine = ´EDXX9P1`
     AND logon_time > SYSDATE – 1;
```

Considerations for Dynamic Performance Views

a. These views are owned by the SYS user.

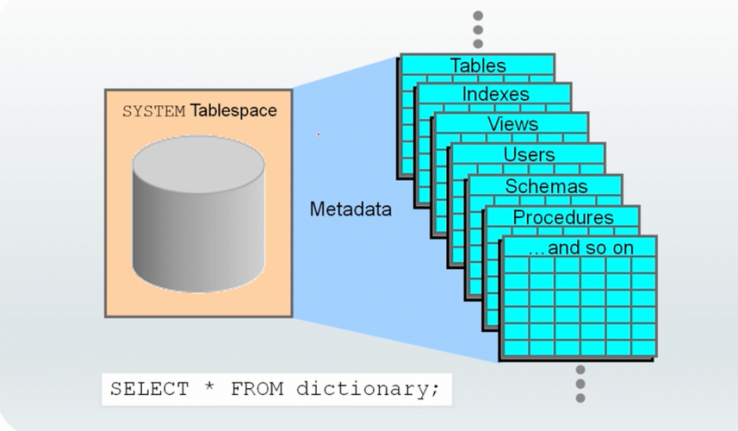
b. Views provide information depending on the stage (NOMOUNT, MOUNT, or OPEN).

c. You can query V$FIXED_TABLES to see all the view names.

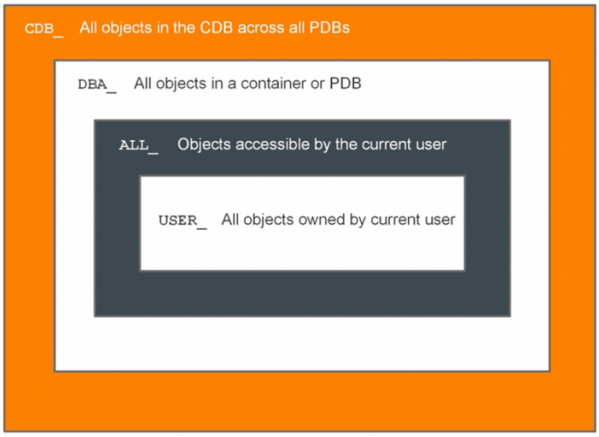
d. These views are often referred to as "v$ views."

e. Read consistency is not guaranteed on these views because the data is dynamic.

Data dictionary: Overview



Querying the Oracle Data Dictionary



Summary

In this lesson, you have learned how to:

- Describe initialization parameter files and initialization parameters.
- View and modify initialization parameters in SQL*Plus
- Work with the Automatic Diagnostic Repository (ADR)
- Query dynamic performance views

I. Investigating Initialization Parameter Files

Overview

In this practical work, you investigate how the Oracle Database server uses initialization parameter files to start the database instance.

Assumptions

You are logged in as the oracle user.

The orclcdb database instance has been started.

Tasks

1. Use oraenv to set the environment variables for the orcl (orclcdb) database.

```
$ . oraenv
ORACLE_SID= [orclcdb] ? orcl
The Oracle base remains unchanged with value /u01/app/oracle
$
$ sqlplus / s sysdba
SQL> define _editor = gedit
```

```
> Set ORACLE_SID=orcl
>
```

2. Start SQL*Plus and connect to the root container as the SYS user with the SYSDBA privilege.

```
$ sqlplus/ as sysdba
…
SQL>
```

3. Locate the default SPFILE for your database instance by using the SHOW PARAMETER command. The results show that the SPFILE is in the ORACLE_HOME/dbs directory (on Ms Windows: ORACLE_HOME/database) . The output in the code box has been formatted for legibility. <version_number> refers to database software version.

```
SQL> SHOW PARAMETER spfile

NAME TYPE VALUE
--------- -------- ----------------------------------------------------------------------------------------
spfile   string   /001/app/oracle/product/<version_number>/dbhome_1/dbs/spfileorclcdb.ora
```

```
NAME          TYPE          VALUE
------------------ ---------------- ------------------------------------------------------------
spfile            string            C:/APP/Admin/db_home/database/spfileorcl.ora
```

4. View the init.ora file. This is the sample text initialization parameter file (PFILE) provided with the Oracle Database installation.
   a. Use the SQL*Plus HOST command to return to the operating system prompt.

```
SQL> host
$
```

```
SQL> host
>
```

b. Change to the $ORACLE_HOME/dbs (Ms Windows: $ORACLE_HOME/database) directory and use the ls command to list the contents of the directory.

```
$ cd $ORACLE_HOME/dbs
$ ls
Hc_CDBDEV.dat     init.ora          orapwCDBTEST      spfileorclcdb.ora
Hc_CDBDEV.dat     1kCDBDEV          orapworclcdb      spfileorclcdb.ora
Hc_orclcdb.dat    1kCDBTEST         orapwrcatedb
hc_rcatcdb.dat    1kORCLCDB         snapcf_orclcdb.f
initCDBDEV. ora   1KRCATCDB         spfileCDBTEST.ora
```

```
> cd $ORACLE_HOME/database
> dir
Hc_CDBDEV.dat     init.ora          orapwCDBTEST      spfileorclcdb.ora
Hc_CDBDEV.dat     1kCDBDEV          orapworclcdb      spfileorclcdb.ora
Hc_orclcdb.dat    1kCDBTEST         orapwrcatedb
hc_rcatcdb.dat    1kORCLCDB         snapcf_orclcdb.f
initCDBDEV. ora   1KRCATCDB         spfileCDBTEST.ora
```

Notice that in Linux: the SPFILE (spfileorclcdb.ora) and init.ora files are stored in dbs directory. The naming convention for an SPFILE is spfile<SID>.ora.
in Windows machine: the SPFILE (spfileorclcdb.ora) file is stored in database directory. The naming convention for an SPFILE is spfile<SID>.ora.

c. Use the cat or more command to view the contents of the sample text initialization parameter file (PFILE), init.ora. Then exit from the HOST shell back to SQL*Plus.

```
$ more init.ora
…
$ exit
```

d. The initorcl.ora file does not exist. So, create it in SQL*Plus.

```
SQL> CREATE PFILE = '$ORACLE_HOME/dbs/initorcl.ora' FROM SPFILE;
File created.
SQL> ! --The '!' character is a shortcut for the host command
$ cd $ORACLE_HOME/dbs
$ ls
```

```
SQL> CREATE PFILE = '$ORACLE_HOME/database/initorcl.ora' FROM SPFILE;
File created.
SQL> ! --The '!' character is a shortcut for the host command
$ cd $ORACLE_HOME/database
```

e. Now use the cat or more command to view the text initialization parameter file, initorcl.ora.

```
$ more initorcl.ora
….
$
```

f. Return to SQL*Plus.

```
$ exit
SQL>
```

5. If the database server doesn't find an SPFILE, then the text initialization parameter file will be used. Now you'll set up a test to see how the search works when you start the database instance.

a. Shut down the database instance in IMMEDIATE mode.

```
SQL> SHUTDOWN IMMEDIATE
Database closed.
Database dismounted.
SQL>
```

b. Use the HOST command or ! to return to an operating system prompt.

```
SQL> host
$
```

c. Change to the $ORACLE_HOME/dbs directory.

```
$ cd $ORACLE_HOME/dbs
$
```

```
$ cd $ORACLE_HOME/database
$
```

d. Rename the spfileorcl.ora file to orig_spfileorcl.ora. Renaming this file will take it out of the search order for parameter files when you start up the database instance. Instead, the database server will automatically find the initORCL.ora file (PFILE) to start the database instance.

```
$ mv spfile orcl.ora orig_spfileorcl.ora
$
```

```
$ move spfile_orcl.ora orig_spfileorcl.ora
$
```

e. Return to SQL*Plus.

```
$ exit
SQL>
```

f. Start the database instance by using the STARTUP command.

```
SQL> STARTUP
SQL>
```

g. Verity that the database instance was started with your PFILE by issuing the SHOW PARAMETER spfile command. The value is null, which means the database instance was started with a PFILE.

```
SQL> SHOW PARAMETER spfile
NAME                 TYPE        VALUE
-------------------- ----------- ------------------
```

```
spfile              string
SQL>
```

6. Configure the database instance to start with the SPFILE once again.
   a. Shut down the database instance in IMMEDIATE mode.

```
SQL> SHUTDOWN IMMEDIATE
Database closed.
Database dismounted.
ORACLE instance shut down.
SQL>
```

   b. Use the HOST command to return to the operating system.

```
SQL> host
$
```

   c. Change to the $ORACLE_HOME/dbs directory.

```
$ cd $ORACLE_HOME/dbs
$
```

```
$ cd $ORACLE_HOME/database
$
```

   d. **Rename** the orig_spfileorcl.ora file to spfileorcl.ora.

```
$ mv orig_spfileorcl.ora spfileorcl.ora
$
```

```
$ move orig_spfileorcl.ora spfileorcl.ora
$
```

   e. Return to SQL*Plus.

```
$ exit
SQL>
```

   f. Start the database instance by using the STARTUP command.

```
SQL> STARTUP
SQL>
```

Renaming this file will take it out of the search order for parameter files when you start up the database instance. Instead, the database server will automatically find the initORCL.ora file (PFILE) to start the database instance.

   g. Verity that the database instance was started with your PFILE by issuing the SHOW PARAMETER spfile command. The value is null, which means the database instance was started with a PFILE.

```
SQL> SHOW PARAMETER spfile

NAME          TYPE  VALUE
------------- ------ ------------------------------------------------------------------------------
spfile        string   /u01/app/oracle/product/<version_number/>/dbhome_1/dbs/spfileorcl.ora

SQL>
```

```
SQL> SHOW PARAMETER spfile

NAME                 TYPE        VALUE
-------------------- ----------- ------------------------------------------ --------------------
spfile               string      C:/app//ADMIN/db_home/database/spfileorcl.ora

SQL>
```

7. Exit SQL*Plus.

```
SQL> EXIT
$
```

II. Viewing Initialization Parameters by Using SQL*Plus

Overview

In this part, you view initialization parameters (parameters) by using SQL*Plus. You do this in two ways:

- By using the SHOW PARAMETER command
- By querying the following views: V$PARAMETER, V$SPPARAMETER, V$PARAMETER2, and V$SYSTEM_PARAMETER

Assumptions

You are logged in as the oracle OS user.

Tasks

View Basic Parameters

In this section, you view basic parameters by using the SHOW PARAMETER command. Basic parameters are those parameters that you are likely to modify.

1. Be sure your environment is using the orcl database.

```
$ . oraenv
…
```

2. Start SQL*Plus and connect to the root container as the SYS user with the SYSDBA privilege.

```
$ sqlplus/ as sysdba
..
SQL>
```

3. View the values of the DB_NAME and DB_DOMAIN parameters. Together, these values create the global database name.
   a. View the value of the DB_NAME parameter. This parameter specifies the current database identifier of up to eight characters. If you have multiple databases, the value of this parameter should match the Oracle instance identifier of each one to avoid confusion with other databases running on the system.

```
SQL> SHOW PARAMETER db_name
NAME                                 TYPE                                       VALUE
------------------------------------ ------------------------------------------ ----------
```

```
db_name                              string                               orcl
SQL>
```

b. View the value of the DB_DOMAIN parameter. In a distributed database system, DB_DOMAIN specifies the logical location of the database within the network structure. You should set this parameter if this database is or ever will be part of a distributed system. There is no default value.

```
SQL> SHOW PARAMETER db_domain
NAME                                 TYPE                 VALUE
------------------------------------ -------------------- ---------
db_domain                            string
SQL>
```

4. View the DB_RECOVERY_FILE_DEST and DB_RECOVERY_FILE_DEST_SIZE parameters. These parameters set the location of the fast recovery area and its size.

The DB_RECOVERY_FILE_DEST parameter specifies the default location for the fast recovery area. The fast recovery area contains multiplexed copies of current control files and online redo logs, as well as archived redo logs, flashback logs, and Recovery Manager (RMAN) backups. If you specify a value for DB_RECOVERY_FILE_DEST, you must also specify a value for the DB_RECOVERY_FILE_DEST_SIZE initialization parameter.

The DB_RECOVERY_FILE_DEST_SIZE parameter specifies (in bytes) the hard limit on the total space to be used by target database recovery files created in the fast recovery area.

```
SQL> SHOW PARAMETER db_recovery file_dest
NAME                                 TYPE                 VALUE
------------------------------------ -------------------- ------------------------------
db_recovery_file_dest                string                u01/app/oracle/fast_recovery_area

db_recovery_file_dest_size           big integer           14970M
SQL>
```

```
SQL> SHOW PARAMETER db_recovery file_dest
NAME                                 TYPE                 VALUE
------------------------------------ -------------------- ------------------------------
db_recovery_file_dest                string                C:/app/oracle/fast_recovery_area

db_recovery_file_dest_size           big integer           14970M
SQL>
```

5. View the SGA_TARGET and SGA_MAX_SIZE parameters.

SGA_TARGET specifies the total amount of SGA memory available to a database instance. and SGA_MAX_SIZE sets a maximum size for the SGA.

If you set the SGA_TARGET parameter, you enable the Automatic Shared Memory Management (ASMM) feature. The Oracle Database server will automatically distribute memory among the

various SGA memory pools (buffer cache, shared pool, large pool, java pool, and streams pool), ensuring the most effective memory utilization. Note, the log buffer pool, other buffer caches (such as KEEP and RECYCLE), other block sizes, fixed SGA, and other internal allocations must be manually sized and are not affected by ASMM. The memory allocated to these pools is deducted from the total available memory for SGA_TARGET when ASMM is enabled.
The manageability monitor process (MMON) computes the values of the automatically tuned memory pools to support ASMM.

In addition to SGA_TARGET and SGA_MAX_SIZE, you can set minimum nonzero values for each memory pool if an application component needs a minimum amount of memory to function properly. ASMM will treat those values as minimum levels.
The range of values for SGA_TARGET can be from 64MB to an operating system-dependent value.

```
SQL> SHOW PARAMETER sga
NAME                                 TYPE                              VALUE
------------------------------------ --------------------------------- ----------
allow_group_access_to_sga            Boolean                           FALSE
lock_sga                             Boolean                           FALSE
pre_page_sga                         Boolean                           TRUE
sga_max_size                         big integer                       1920M
sga_min_size                         big integer                       0
sga_target                           big integer                       1920M
unified_audit_sga_queue_size         integer                           1048576
SQL>
```

6. View the UNDO_TABLESPACE parameter. This parameter specifies the undo tablespace to be used when an instance starts. Oracle Database creates and manages information that is used to roll back, or undo, changes to the database. Such information consists of records of the actions of transactions, primarily before they are committed. These records are collectively referred to as undo and are stored in the undo tablespace. The results below indicate that the undo tablespace in your environment is UNDOTBS1.

```
SQL> SHOW PARAMETER undo_tablespace

NAME                                 TYPE                                          VALUE
------------------------------------ --------------------------------------------- --------------
Undo_tablespace                      string                                        UNDOTBS1

SQL>
```

7. View the UNDO_TABLE parameter. This parameter specifies the release with which Oracle must maintain compatibility. It enables you to use a new release of Oracle, while at the same time guaranteeing backward compatibility with an earlier release. This is helpful if it becomes necessary to revert to the earlier release. By default, the value for the compatible entry for this parameter is equal to the version of the Oracle Database that you have installed.

```
SQL> SHOW PARAMETER compatible
```

```
NAME                                 TYPE                                 VALUE
------------------------------------ ------------------------------------ ----------
compatible                           string                               19.0.0
noncdb_compatible                    boolean                              FALSE

SQL>
```

8. View the CONTROL_FILES Initialization parameter. This parameter specifies one or more control files, separated by commas, and including paths. One to **eight** file names are listed. Oracle strongly recommends that you multiplex and mirror control files. The output has been formatted for legibility.

```
SQL> SHOW PARAMETER control_files

NAME                                 TYPE                                 VALUE
------------------------------------ ------------------------------------ ------------------------------
control files                        string                               /u01/app/oracle/oradata/ORCL
                                                                          /control01.ctl, /u01/app/oracle/
                                                                          fast_recovery_area/ORCL/control02.ctl

SQL>
```

```
SQL> SHOW PARAMETER control_files

NAME                                 TYPE                 VALUE
------------------------------------ -------------------- ----------------------------------------------
control files                        string               C:/app/oracle/oradata/ORCL
                                                          /control01.ctl, C:/app/oracle/
                                                          fast_recovery_area/ORCL/control02.ctl

SQL>
```

9. View the PROCESSES, SESSIONS, and TRANSACTIONS initialization parameters.
   a. View the PROCESSES parameter. This parameter specifies the maximum number of operating system **user processes** that can simultaneously connect to an Oracle server. This value should allow for all background processes and user processes. The default values of the SESSIONS and TRANSACTIONS initialization parameters are derived from the PROCESSES parameter. Therefore, if you change the value of PROCESSES, you should evaluate whether to adjust the values of those derived parameters. The range of values is from six to an OS-dependent value. The default value is dynamic and dependent on the number of CPUs.

```
SQL> SHOW PARAMETER processes
NAME                                 TYPE                                 VALUE
```

```
---------------------------------------------------- ------------------------------------ ----------
---
aq_tm_processes                                      integer                                 1
db_writer processes                                  integer                                 1
gcs_server_processes                                 integer                                 0
global_txn_processes                                 integer                                 1
job_queue_ processes                 integer                          40
log_archive_max_processes                            integer                                 4
processes                                            integer                                 300
SQL>
```

b. View the SESSIONS parameter. This parameter specifies the maximum number of sessions that can be created in the system. Because every login requires a session, this parameter effectively determines the maximum number of concurrent users in the system. Notice in the results that the session entry has a value of 472. You should always set this parameter explicitly to a value equivalent to your estimate of the maximum number of concurrent users, plus the number of background processes, plus approximately 10% for recursive sessions.

```
SQL> show parameters sessions

NAME                                 TYPE                 VALUE
------------------------------------ -------------------- ----------
java_max_sessionspace_size           integer              0
java_soft sessionspace_limit         integer              0
license_max_sessions                 integer              0
license_sessions_warning             integer              0
sessions                             integer              472
shared_server_sessions               integer

SQL>
```

View Advanced Parameters

In this section, you use the SHOW PARAMETER command to view advanced parameters.

10. View the TRANSACTIONS parameter. This is an advanced parameter and seldom needs any adjustment. This parameter specifies how many rollback segments to bring online when the UNDO MANAGEMENT initialization parameter is equal to MANUAL. A transaction is assigned to a rollback segment when the transaction starts, and it can't change for the life of the transaction. A transaction table exists in the rollback segment header with limited space, limiting how many transactions a single segment can support. Therefore, X number of concurrent transactions require at least Y number of rollback segments. With Oracle Automatic Undo Management, the database creates rollback segments, brings them online, takes them offline, and drops them as needed.

```
SQL> SHOW PARAMETER transactions

NAME                                 TYPE                 VALUE
------------------------------------ -------------------- ----------
```

```
Transactions                          integer                        519
Transactions_per_rollback_segment     integer                        5

SQL>
```

11. View the configuration for the DB_FILES initialization parameter. This parameter specifies the maximum number of database files that can be opened for this database. The range of values is OS-dependent.

```
SQL> SHOW PARAMETER db_files

NAME                                 TYPE                           VALUE
------------------------------------ ------------------------------ ----------
db_files                             integer                        200

SQL>
```

12. View the COMMIT_LOGGING parameter. This parameter is used to control how redo is batched by the Log Writer process. There is no default value, as shown below. You can modify this parameter in a PDB.

```
SQL> SHOW PARAMETER commit_logging
```

13. View the COMMIT_WAIT parameter. This parameter is used to control when the redo for a commit is flushed to the redo logs. There is no default value.

```
SQL> SHOW PARAMETER commit_wait
```

14. View the SHARED_POOL_SIZE parameter. This parameter specifies the size of the shared pool in bytes. The shared pool contains objects such as shared cursors, stored procedures. control structures, and parallel execution message buffers. The range of values is OS-dependent. The default value is zero if the SGA_TARGET parameter is set. Otherwise, the value is 128 MB for a 64-bit platform or 48 MB for a 32-bit platform.

```
SQL> SHOW PARAMETER shared_pool_size
```

15. View the DB_BLOCK_SIZE parameter. This parameter specifies the standard Oracle database block size (in bytes) and is used by all tablespaces by default. Its value is set during database creation and cannot be subsequently changed. The range of values is from 2048 to 32768 (OS-dependent). The default value is 8192.

```
SQL> SHOW PARAMETER db_block_size
```

16. View the DB_CACHE_SIZE initialization parameter. You configure this parameter to specify the size of the standard block buffer cache (default buffer pool). The range of values is at least 4 MB times the number of CPUs. Smaller values are automatically rounded up to this value. The default value is zero if the SGA_TARGET initialization parameter is set, otherwise the larger of 48 MB or (4 MB*CPU_COUNT).

```
SQL> SHOW PARAMETER db_cache_size
```

17. View the UNDO_MANAGEMENT parameter. This parameter specifies the undo space management mode that the system should use. When set to AUTO, the instance is started

in automatic undo management mode. Otherwise, it is started in rollback undo mode. In rollback undo mode, undo space is allocated as rollback segments. In automatic undo mode, undo space is allocated as undo tablespaces. The value is AUTO or MANUAL If the UNDO_MANAGEMENT parameter is omitted when the instance is started, the default value AUTO is used.

```
SQL> SHOW PARAMETER undo_management
```

18. View the MEMORY_TARGET and MEMORY_MAX_TARGET parameters.
    MEMORY_TARGET specifies the Oracle system-wide usable memory. The database server tunes memory to the MEMORY_TARGET value, reducing or enlarging the SGA and PGA as needed. MEMORY_MAX_TARGET sets a maximum value for MEMORY_TARGET.

In a PFILE, if you omit MEMORY_MAX_TARGET and include a value for MEMORY_TARGET. the database automatically sets MEMORY_MAX_TARGET to the value of MEMORY_TARGET. I you omit the line for MEMORY_TARGET and include a value for MEMORY_MAX_TARGET, the MEMORY_TARGET parameter defaults to zero. After startup, you can dynamically change MEMORY_TARGET to a nonzero value if it does not exceed the value of MEMORY_NAX_TARGET. For NEMORY_TARGET. values range from 152 MB to MEMORY_MAX_TARGET.

a. View the MEMORY_TARGET parameter.

```
SQL SHOW PARAMETER memory_target
```

b. View the MEMORY_MAX_TARGET parameter.

```
SQL> SHOW PARAMETER memory_max_target
```

19. View the PGA_AGGREGATE_TARGET parameter. This parameter specifies the amount of Program Global Area (PGA) memory available to all server processes attached to the database instance. This memory does not reside in the System Global Area (SGA). The database uses this parameter as a target amount of PGA memory to use. When setting this parameter, subtract the SGA from the total memory on the system that is available to the Oracle instance. The minimum value is 10 MB, and the maximum value is 4096 GB minus 1. The default value is 10 MB or 20% of the size of the SGA, whichever is greater.

```
SQL SHOW PARAMETER pga_aggregate_target
```

Query Views for Parameter Values

In this section, you query views to learn about parameters.

20. Query the data dictionary to find views that contain the word "parameter." The query below returns 66 rows. Not all these views contain information about initialization parameters. Among these rows are the v$PARAMETER, V$SPPARAMETER, V$PARAMETER2, and V$SYSTEM_PARAMETER views, which you'll examine next.

```
SQL> SET PAGES 100
SQL> SELECT table_name FROM dict WHERE table_name LIKE '%PARAMETER%';
```

```
TABLE_NAME
--------------------------------------------------------------------------------------------------------
USER_ADVISOR EXEC PARAMETERS
USER_ADVISOR_PARAMETERS
…
V$PARAMETER
V$ PARAMETER_VALID_VALUES
V$SYSTEM_RESET_PARAMETER2
V$SPPARAMETER
V$SYSTEM_PARAMETER
V$SYSTEM_PARAMETER2
V$SYSTEM_RESET_PARAMETER

66 rows selected.
SQL>
```

21. Explore the VSPARAMETER view. This view displays the current parameter values in the current session.
    a. View the columns in the VSPARAMETER view by using the DESCRIBE command. This command returns column names, whether null values are allowed (NOT NULL is displayed if the value cannot be null), and column data types.

The results below contain a column named ISSYS_MODIFIABLE. This column is important because it tells you whether a parameter is static or dynamic. If its value is FALSE, then the parameter is static; otherwise, it's dynamic. To change a static parameter, you must shut down and restart the database; however, you can modify a dynamic parameter in real time while the database is online.

```
SQL> DESCRIBE v$parameter
```

    b. Query NAME, ISSYS_MODIFIABLE, and VALUE in the VSPARAMETER view. The query returns many rows.

The TRANSACTIONS parameter is static as indicated by FALSE in the ISSYS_MODIFIABLE column. The PLSQL_WARNINGS parameter is dynamic as indicated by IMMEDIATE in the ISSYS_MODIFIABLE column.

Optional: Before entering the following command, you can enter SET PAUSE ON to cause a pause after each page output. Press Enter to display each next page. After all pages have been displayed, you can issue the SET PAUSE OFF command to stop this feature.

```
SQL> COLUMN name FORMAT A35
SQL> COLUMN value FORMAT A20
SQL> SELECT name, issys_modifiable, value FROM v$parameter order by name;

NAME                                ISSYS_MOD                      VALUE
----------------------------------- ------------------------------ ----------------------
DBFIPS_140                          FALSE                          FALSE
active_instance_count               FALSE                          …
Transactions                        FALSE                          519
```

```
transactions_per_rollback_segment   FALSE                    5
…
wallet_root                         FALSE
workarea_size_policy                IMMEDIATE              AUTO
xml_db_events                       IMMEDIATE              enable

445 rows selected.
SQL>
```

c. Query the V$PARAMETER View again, but this time be more specific. Include a WHERE clause to specify all parameters that contain the word "pool." The query returns all parameters that contain the string "pool."

Note: The values shown may vary from the values displayed in the output.

```
SQL> SELECT name, value FROM v$parameter
WHERE name LIKE '%pool%';

NAME                                                                 VALUE
-------------------------------------------------------------------- ----------
shared_pool_size                                                     0
large_pool_size                                                      0
java_pool_size                                                       0
streams_pool_size                                                    0
shared_pool_reserved_size                                            52848230
memoptimize pool_size                                                0
buffer_pool_keep
buffer_pool_recycle
olap_page _pool_size                                                 0

9 rows selected.
SQL>
```

22. Explore the VSSPPARAMETER view. This view contains information about the contents of the server parameter file. If a server parameter file was not used to start the instance, each row of the view will contain FALSE in the ISSPECIFIED column.

a. View the columns in the VSSPPARAMETER View by using the DESCRIBE command.

```
SQL> DESCRIBE v$spparameter
```

b. Query NAME and VALUE in the VSSPPARAMETER view. Browse the rows returned by the query.

```
SQL> SELECT name, value FROM v$spparameter;
```

23. Explore the VSPARAMETER2 view. This view contains information about the initialization parameters that are currently in effect for the session. For parameters with more than one value assigned such as the control_files parameter, each parameter value will be listed as a row in the view's output. A new session inherits parameter values from the instance wide values displayed in the VSSYSTEM_PARAMETER2 view.

a. View the columns in the V$PARAMETER2 View by using the DESCRIBE command.

```
SQL> DESCRIBE v$parameter2
```

b. Query NAME and VALUE in the V$PARAMETER2 view. Browse the rows returned by the query.

```
SQL> SELECT name value FROM v$parameter2;
```

24. Explore the V$SYSTEM_PARAMETER View. This view contains information about the initialization parameters that are currently in effect for the instance.
    a. View the columns in the V$SYSTEM_PARAMETER View by using the DESCRIBE command.

```
SQL> DESCRIBE v$system_parameter
```

b. Query NAME and VALUE in the V$SYSTEM_PARAMETER view. Browse the rows returned by the query.

```
SQL> SELECT name, value FROM v$system_parameter;
```

25. 25. Exit SQL*Plus.

```
SQL> EXIT
$
```

III. Modifying Initialization Parameters by Using SQL*Plus

Overview

In this practice, you modify the following kinds of initialization parameters (parameters) with SQL*Plus:

- Session-level parameter
- Dynamic system-level parameter
- Static system level parameter

Assumptions

You are connected to the compute node as the oracle user.

The Oracle environment is set to access the orcl instance.

Tasks

Modify a Session-Level Parameter

In this section, you modify the NLS_DATE_FORMAT parameter. This parameter defines the default date format to use with the TO_CHAR and TO_DATE functions. The NLS_TERRITORY parameter determines the default value of NLS_DATE_FORMAT. NLS_DATE_FORMAT is one of the National Language Support (NLS) parameters that you can customize just for your session, therefore making it a session-level parameter. When your session ends, your modification expires, and the parameter is returned to as default value.

1. Start SQL*Plus and log in to the database as the sys user with the SYSDBA privilege.

```
$ sqlplus / as syadba
…
SQL>
```

2. Learn about the NLS_DATE_FORMAT parameter by querying the VSPARAMETER view. Include a WHERE clause to narrow down the query to just the NLS_DATE_FORMAT parameter. Remember that in the V$PARAMETER view, the **parameter names are in lowercase**.

```
SQL> column VALUE Format A40
SQL> column NAME FORMAT A40
SQL> SELECT name, isses_modifiable, issys_modifiable, ispdb_modifiable, value FROM
v$parameter WHERE name= 'nls_date_format';

NAME                 ISSESS         ISSYS_MOD          ISPDB VALUE
-------------------- -------------- ------------------ ---------- ----------
nls_date_format      TRUE           FALSE              TRUE
```

3. Find out the default date format for the database by querying the NLS_TERRITORY parameter in the VSPARAMETER view. Include a WHERE clause to narrow down the query to just the NLS_TERRITORY parameter. Remember that in the VSPARAMETER view, the parameter names are in lowercase.

Note: NLS_TERRITORY is set at installation and can be changed with ALTER SESSION.

```
SQL> SELECT name, value FROM v$parameter WHERE name = 'nls_territory';

NAM                              VALUE
-------------------------------- ---------------
nls_territory                    AMERICA

SQL>
```

4. Connect to ORCLPDB. Run a simple query against the sample data to view an example of the current default date format in use.
    a. Switch to ORCLPDB by using the ALTER SESSION command.

```
SQL> ALTER SESSION SET container = ORCLPDB;
Session altered.

SQL>
```

    b. Query the LAST_NAME and HIRE_DATE Columns in the HR.EMPLOYEES table. Notice the date format is dd-mon-rr.

```
SQL> SELECT last_name, hire_date FROM hr.employees;

LAST_NAME                                HIRE_DATE
---------------------------------------- -----------------
King                                     17-JUN-03
Kochhar                                  21-SEP-05
…
Higgins                                  07-JUN-02
```

```
Gietz                                   07-JUN-02

107 rows selected.
SQL>
```

5. Modify the NLS_DATE_FORMAT parameter to use the format mon dd yyyy by using the ALTER SESSION Command.

```
SQL> ALTER SESSION SET nls_date_format= 'mon dd yyyy';

Session altered.
SQL>
```

6. Rerun the last SELECT query. Notice that the date format has changed from dd mon rr to mon dd yyyy.

```
SQL> SELECT last_name, hire_date FROM hr.employees;

LAST_NAME                                          HIRE_DATE
-------------------------------------------------- -----------------
King jun                                           17 2003
Kochhar                                            sep 21 2005
…
Higgins
Gietz

107 rows selected.
SQL>
```

7. Query the NLS_DATE_FORMAT parameter again by using the SHOW PARAMETER command. The value column now reflects the custom date format.

```
SQL> SHOW PARAMETER nls_date_format

NANE                             TYPE                VALUE
-------------------------------- ------------------- -----------------
Nls_date_format                  string              mon dd yyyy

SQL>
```

8. Disconnect from ORCLPDB to end your session.

```
SQL> DISCONNECT
```

9. Connect to ORCLPDB again as the SYSTEM user by using the Easy Connect syntax

The easy connect syntax is //<full hostname>: <port number>/<service name>.

a. Get the full hostname.

```
SQL> ! hostname -f
```

b. Get the listener port number. The grep command reduces the lines to just those using TCP or TCPS protocol. Look for the port using TCP.

```
SQL> ! lsnrctl status | grep tcp l grep PORT
(DESCRIPTION= (ADDRESS (FROTOCOL=tcp) (HOST=edvmr1p0.us.oracle.com )
(PORT=1521)))
(DESCRIPTION= (ADDRESS=(PROTOCOL=tcps) (HOST=edvarlp0.us.oracle.com)
(PORT=5502)) (Security=(my_wallet_directory=/u01/app/oracle/admin/CDBTEST/xdb_wallet))
(Presentation-HTTP) (Session=RAW))
(DESCRIPTION (ADDRESS=(PROTOCOL-tcps) (HOST=edvmrlpo.us.oracle.com) (PORT-
5500)) (Security- (my_wallet_directory-/u01/app/oracle/admin/orclcdb/xdb_wallet))
(Presentation-HTTP) (Session=RAW))
```

Chercher les commandes qui donnent le nom de machine et celle qui donne le port d'écoute du listener sous windows

c. Connect as a user with SYSDBA privileges.

```
SQL> connect / as sysdba
```

d. Query the VSSERVICES view. Discover the network name.

Note: The network name is the name of the fully qualified database instance and is formed by default from the instance_name and db_domain_name. In this practice environment, the network name is the same as the database service name.
Here are some of the affected columns present in the V$SERVICES view:
SERVICE_ID: The service identifier.
NAME: The name of the service.
NETWORK_NAME: The name of the service used for name resolution.
CREATION_DATE: The creation date of the service.
FAILOVER_METHOD: The failover method used by the service in the event of a failure.

```
SQL> SELECT name, network_name FROM V$SERVICES;
WHERE name = 'orclpdb';

NAME                             NETWORK NAME
-------------------------------- ------------------------
orclpdb                          orclpdb
```

e. Using values from steps 9-a through 9-d, construct and use the Easy Connect string to connect to the orclpdb PDB. Replace <service_name> in the connect command with the name value from the previous step.

Note: The '//' characters preceding the <full hostname> are optional.

```
SQL> connect system/<password>@//<full hostname>: <port number>/<service name> ===>
SQL> connect system/emsi@//localhost:1521/orclpdb
Connected.
SQL>
```

10. Rerun the query against the HR.EMPLOYEES table. The date format has reverted to the default format dd mon rr. A session-level parameter change only lasts for the duration of the session. A connect command creates a new session.

```
SQL> SELECT last_name, hire_date FROM hr.employees;
LAST NAME                      HIRE_DATE
------------------------------ -----------------
…
Higgins                        07-JUN-02
Gietz                          07-JUN-02

107 rows selected.
SQL>
```

11. Query the NLS_DATE_FORMAT parameter again by using the SHOW PARAMETER command The VALUE column no longer has the custom date format.

```
SQL> SHOW PARAMETER nls_date_format

NAME                                 TYPE                              VALUE
------------------------------------ --------------------------------- ----------
nls_date_format                      string

SQL>
```

Modify a Dynamic System-Level Parameter

In this section, you modify the JOB_QUEUE_PROCESSES parameter. This parameter specifies the maximum number of job slaves per database instance that can be created for the execution of DBMS_JOB jobs and Oracle Scheduler (DBMS_SCHEDULER) jobs.

1. Exit SQL*Plus and connect to the root container with the SYSDBA privilege. If you try to update the JOB_QUEUE_PROCESSES parameter from a PDB, you'll get an error. Also, you'll need the SYSDBA privilege to restart the database instance later.

```
SQL> exit
…
$ sqlplus / as sysdba
…
SQL>
```

2. Learn about the JOB_QUEUE_PROCESSES parameter by querying the VSPARAMETER view. Include a WHERE clause to narrow down the query to just the JOB_QUEUE_PROCESSES parameter. Remember that in the VSPARAMETER view, the parameter names are in lowercase.

```
SQL> column name format A20
SQL> column value format A20
```

```
SQL SELECT name, isses_modifiable, issys_modifiable, value FROM V$parameter WHERE
name = 'job_queue_processes';

NAME                                               ISSES          ISSYS_MOD            VALUE
-------------------------------------------------- -------------- -------------------- ---------
job-queue-processes                                FALSE          IMMMEDIATE           80
SQL>
```

3. Change the JOB_QUEUE_PROCESSES parameter value to 15 by using the ALTER SYSTEM command. Set SCOPE equal to BOTH so that the change happens in both the database *instance memory* (which makes the change immediate) and in the *SPFILE* (which makes the change permanent).

```
SQL> ALTER SYSTEM SET job_queue_processes=15 SCOPE=BOTH;
system altered.

SQL>
```

4. Use the SHOW_PARAMETER Command to verify that the JOB_QUEUE_PROCESS value is now equal to 15. Notice that only job was entered with the SHOW_PARAMETER command instead of the full name, job_queue_processes. Remember, when you use SHOW PARAMETER command, you don't have to enter the full name. The database server will find all parameters that contain the letters job. In this example, the database server found three parameters that contain the letters job. The query result indicates that the job_queue_processes value in memory is now 15.

```
SQL> SHOW PARAMETER Job

NAME                                 TYPE                 VALUE
------------------------------------ -------------------- -----------
job_queue_processes                  integer              15
max_datapump_jobs_per_pdb            integer              100
max_datapump_parallel_per_job        string               50

SQL>
```

5. Verify that the new value for the JOB_QUEUE_PROCESSES parameter persists after the database instance is restarted.
   a. Shut down the database instance with the IMMEDIATE mode.

```
SQL> SHUTDOWN IMMEDIATE
Database closed.
Database dismounted.
ORACLE instance shut down.

SQL>
```

   b. Start the database instance by using the STARTUP command.

```
SQL> STARTUP
…
SQL>
```

c. View the configuration for the JOB_QUEUE_PROCESSES parameter again by using the SHOW PARAMETER command. The value is 15, which proves that your change to the parameter persisted after the database instance was restarted.

```
SQL> SHOW PARAMETER job
```

Modify a Static System-Level Parameter

In this section, you modify the SEC_MAX_FAILED_LOGIN_ATTEMPTS parameter. This parameter specifies the number of authentication attempts that can be made by a client on a connection to the server process. These login attempts can be for multiple user accounts in the same connection. After the specified number of failure attempts, the connection will be automatically dropped by the server process.

1. Learn about the SEC_MAX_FAILED_LOGIN_ATTEMPTS parameter by querying the V$PARAMETER view. Include a WHERE clause to narrow down the query to just the SEC_MAX_FAILED_LOGIN_ATTEMPTS parameter. Remember that in the VSPARAMETER view, the parameter names are in lowercase. The query results below have been formatted for easier viewing.

```
SQL> SELECT name, isses_modifiable, issys_modifiable, value FROM v$parameter WHERE
name= 'sec_max_failed_login_attempts';

NAME                                 ISSES           ISSYS_MOD          VALUE
------------------------------------ ----------------- ------------------------- ----------
sec_max_failed_login_attempts        FALSE           FALSE              3

SQL>
```

2. Change the SEC_MAX_FAILED_LOGIN_ATTEMPTS parameter value to 2 by using the ALTER SYSTEM Command. Include the comment 'Reduce for tighter security' and set the scope equal to SPFILE so that the change is made only in the SPFILE. When you specify SCOPE as SPFILE or as BOTH, an optional COMMENT clause lets you associate a text string with the parameter update. The comment is written to the SPFILE.
   a. What happens if you set SCOPE=BOTH?

```
SQL> ALTER SYSTEM SET sec_max_failed_login_attempts = 2 SCOPE=BOTH;
ERROR at line 1:
ORA-02095: specified initialization parameter cannot be modified.
```

   b. Now set SCOPE=SPFILE and include the comment.

```
SQL> ALTER SYSTEM SET sec_max_failed_login_attempts = 2 COMMENT='Reduce for
tighter security.' SCOPE=SPFILE;
System altered.
```

```
SQL>
```

3. View the SEC_MAX_FAILED_LOGIN_ATTEMPTS parameter value by using the SHOW PARAMETER command. The query result indicates that the value hasn't been updated yet. It's still equal to 3 because you need to restart the database instance for the change to take effect, which is required for static parameters.

```
SQL> SHOW PARAMETER sec_max

NAME                                 TYPE                 VALUE
------------------------------------ -------------------- -----------
sec_max_failed_login_attempts        integer              3

SQL>
```

4. Restart the database and then verify that the new value for the SEC_MAX_FAILED_LOGIN_ATTEMPTS parameter is updated.
   a. Shut down the database instance with the IMMEDIATE mode.

```
SQL> SHUTDOWN immediate
Database closed.
Database dismounted.
ORACLE instance shut down.

SQL>
```

   b. Start the database instance by using the STARTUP command.

```
SQL> STARTUP
…

SQL>
```

   c. View the SEC_MAX_FAILED LOGIN_ATTEMPTS parameter value again by using the SHOW PARAMETER command. The query result indicates that the parameter's value was successfully changed to 2.

```
SQL> SHOW PARAMETER sec_max
NAME                                 TYPE                 VALUE
------------------------------------ -------------------- -----------
sec_max_failed_login_attempts        integer              2
TYPE

SQL>
```

   d. View the NAME and UPDATE_COMMENT Columns in the VSPARAMETER view for the SEC_MAX_FAILED_LOGIN_ATTEMPTS parameter. Notice that the comment you added is stored in this view. The results below are formatted for easier reading.

```
SQL> SELECT name, update_comment
FROM v$parameter WHERE name='sec_max_failed_login_attempts';
```

```
NAME                                                 UPDATE COMMENT
---------------------------------------------------------------- ------------------------
sec_max_failed_login_attempts                        reduce for tighter security.
```

5. Change the SEC_MAX_FAILED_LOGIN_ATTEMPTS parameter value back to its original value.

```
SQL> ALTER SYSTEM SET sec_max_failed_login_attempts = 3 COMMENT=' '
SCOPE=SPFILE;
```

6. Exit SQL*Plus and close the terminal window.

```
SQL> EXIT
```

IV. Viewing Diagnostic Information

Overview

In this practice, you perform the following tasks:

- Examine the structure of the Automatic Diagnostic Repository (ADR)
- View the alert log two ways-first through a *text editor* and then using the Automatic Diagnostic Repository Command Interpreter (*ADRCI*)
- Enable DDL logging and log some DDL statements in the DDL log file.

The alert log is a file that provides a chronological log of database messages and errors. It is automatically created and stored, by default, in the Automatic Diagnostic Repository (ADR) on the database server in the $ORACLE_BASE/diag/rdbms/<db_name>/<SID>/trace directory.
ADRCI is an Oracle command-line utility that enables you to investigate problems, view health check reports, and package and upload first-failure data to Oracle Support. You can also use the utility to view the names of the trace files in the ADR and to view the alert log. ADRCI has a rich command set that you can use interactively or in scripts.
The DDL log file contains one log record for each DDL statement.

Assumptions

You are logged in as the oracle user.

The Oracle environment is set to access the orcldb database instance.

Tasks

View the ADR Directories

The Automatic Diagnostics Repository (ADR) is a hierarchical file-based repository for handling diagnostic information. You can navigate the contents of ADR by using your operating system's command line, file browsing tools, or Oracle's ADR Command Interpreter (ADRCI). ADRCI is preferred for many tasks.
in this section, you locate the XML and text-only versions of the alert log by querying the V$DIAG_INFO view.

1. Start SQL*Plus and log in to the database as the sys user with the SYSDBA privilege.

```
$ sqlplus/ as sysdba
…
SQL>
```

2. View the locations of the various diagnostics directories in the ADR. The results below have been formatted for easier reading.

- The path that corresponds to the Diag Alert entry in the NAME column is for the XML version. This path is /u01/app/oracle/diag/rdbms/orclcdb/orclcdb/alert C:\ admin\app\oracle\diag\rdbms\orcl\orcl\alert.
- The path that corresponds to the Diag Trace entry is for the text-only version. This path is /u01/app/oracle/diag/rooms/orclcdb/oreledb/trace C:\app\oracle\diag\rooms\orcl/orcl\trace.

```
SQL> SELECT name, value FROM v$diag_info;

NAME                            VALUE
------------------------------- --------------------------------------------------------------------------
Diag Enabled                    TRUE
ADR Base                        /001/app/oracle
ADR Home                        /u01/app/oracle/diag/rdbms/orcl/orcl
Diag Trace                      /u01/app/oracle/diag/rdbmms/orcl/orcl/trace
Diag Alert                      /u01/app/oracle/diag/rdbms/orcl/orcl/alert
Diag Incident                   /001/app/oracle/diag/rdbms/orcl/orcl/incident
Diag Cdump                      /u01/app/oracle/diag/rdbms/orcl/orcl/cdump
Health Monitor                  /u01/app/oracle/diag/rdbms/orcl/orcl/hm
Default Trace File              /u01/app/oracle/diag/rdbms/orcl/orcl/trace /orcicdb_ora_8778.trc
Active Problem Count 0
Active Incident Count 0
ORACLE_HOME                     /u01/app/oracle/product/19.3.0/dbhome_1

12 rows selected,
SQL>
```

```
SQL> SELECT name, value FROM v$diag_info;

NAME                            VALUE
------------------------------- --------------------------------------------------------------------------
Diag Enabled                    TRUE
ADR Base                        C:\app
ADR Home                        C:\app\diag\rdbms\orcl\orcl
Diag Trace                      C:\app\diag\rdbms\orcl\orcl\trace
Diag Alert                      C:\app\diag\rdbms\orcl\orcl\alert
Diag Incident                   C:\app\diag\rdbms\orcl\orcl\incident
Diag Cdump                      C:\appd\iag\rdbms\orcl\orcl\cdump
Health Monitor                  C:\app\diag\rdbms\orcl\orcl\hm
Default Trace File              C:\app/diag\rdbms\orclorcl/trace\orcl_ora_8172.trc

Active Problem Count            3
Active Incident Count           22
ORACLE_HOME                     C:\app\db_home

12 rows selected,
```

```
SQL>
```

Exit SQL*Plus.

```
SQL> EXIT
```

3. Use an Editor to View the Alert Log

3.1. View the XML version of the alert log. The log.xml file is the XML version of the alert log.

a. Browse to the /u01/app/oracle/diag/rdbms/orcl/orcl/alert directory.

```
$ cd /u01/app/oracle/diag/rdbms/orcl/orcl/alert
$
```

```
> cd C:\app\diag\rdbms\orcl\orcl\alert
>
```

b. List the contents of the directory. Notice that there is a log.xml file in this directory.

```
$ dir
log.xml
$
```

c. Use cat or more to scroll through the file. Notice that it is a chronological log of messages **about non-default initialization parameters used at startup**, **errors**, **SQL statements**, **and so on**. Oracle Database uses the alert log to keep a record of these events as an alternative to displaying the information on an operator's console.

```
$ more log.xml
…
```

3.2. View the text-only version of the alert log.

a. Change to the /u01/app/oracle/diag/rdbms/orcl/orcl/trace directory.

```
$ cd /u01/app/oracle/diag/rdbms/orcl/orcl/trace
$
```

```
> cd C:\app\diag\rdbms\orcl\orcl\trace
>
```

b. The alert_orcl.log (format is alert SID.log) file is the text-only version. In this directory, you also have server process trace files (TRC files) and trace map files (TRM files). Each server and background process can write to an associated trace file. When a process detects an internal error, it dumps information about the error to its trace file. Trace map files contain structural information about trace files and are used for searching and navigation.

```
$ ls *.log
alert_orcl.log
$
```

```
> dir *.log
```

```
alert_orcl.log
>
```

c. Open the file with an editor or use a command such as tail to view the contents of the alert log.

```
$ tail -500 alert_orcl.log
$
```

```
> tail -500 alert_orcl.log
$
```

Use ADRCI to View the Alert Log

1. Start the ADRCI tool. Recall that you set the Oracle environment variables at the beginning of this practice; however, only the ORACLE_HOME environment variable needs to be set prior to starting ADRCI. If you ever need to set just that one variable, you can do so by entering the following at the command prompt:

   export PATH=$PATH:$ORACLE_HOME/bin.

```
$adrci
…
adrci>
```

2. View the alert log by using the SHOW ALERT command. The SHOW ALERT command opens the alert log file in the vi editor, by default.

```
Adrci> SHOW ALERT
…
```

3. Enter G (uppercase) to move to the bottom of the alert file.

4. Enter /Starting ORACLE/ and press return. Press N (uppercase) to search from the bottom of the file to find the last time the instance was started. The following will be like your alert log.

Note: Here lowercase and uppercase are important because vi distinguishes them unless you ignore them by setting :set ic.

```
2019-10-11 10:18:26.316000 -04:00
Starting ORACLE instance (normal) (OS id: 5867)
……
Sys-V shared memory will be used for creating SGA
……
```

5. Search forward by entering / **ALTER** to find the line that starts with ALTER DATABASE MOUNT. Here lowercase and uppercase are important because vi distinguishes them.

6. Search forward again by entering /**ALTER** to find the line that starts with ALTER DATABASE OPEN. Notice that the stages that the database goes through during startup are MOUNT and OPEN.
7. Exit the vi editor by entering :q and pressing Enter.
8. Exit adrci.

```
Adrci > exit
$
```

Log DDL Statements in the DDL Log File

1. Determine if DDL logging is enabled in ORCLPDB. If not, enable it by setting the value for the ENABLE_DDL_LOGGING initialization parameter to TRUE.
   a. Start SQL*Plus and log in to the database as the Sys user with the SYSDBA privilege.

```
$ sqlplus/ as sysdba
SQL>
```

   b. Switch to PDB1.

```
SQL> ALTER SESSION SET CONTAINER = ORCLPDB;
Session altered.
SQL>
```

   c. Issue the SHOW PARAMETER command to view the value for ENABLE_DDL_LOGGING,

ENABLE_DDL_LOGGING enables or disables the writing of a subset of data definition language (DDL) statements to a DDL log.
The DDL log is a file that has the same format and basic behavior as the alert log, but it only contains the DDL statements issued by the database. The DDL log is created only for the RDBMS component and only if the ENABLE_DDL_LOGGING initialization parameter is set to true. When this parameter is set to false, DDL statements are not included in the DDL log, but they are included in the alert log.
In Oracle Database Cloud Service, ENABLE_DEL_LOGGING is set to TRUE by default. The default value for ENABLE_DDL_LOGGING IS FALSE in non-Cloud installations.

```
SQL> SHOW PARAMETER enable_ddl_logging

NAME                                 TYPE                 VALUE
------------------------------------ -------------------- -----
enable_ddl_logging                   Boolean              FALSE

SQL>
```

   d. If DDL logging was not enabled, enable it for just this session by using the ALTER SESSION command.

```
SQL> ALTER SESSION SET enable_ddl_logging = TRUE;
```

```
Session altered.
SQL>
```

2. Create and drop a table to generate statements that will be logged.

```
SQL> CREATE TABLE TEST (name varchar2 (15)) ;
Table created.

SQL> DROP TABLE TEST;
Table dropped.

SQL>
```

3. Exit SQL Plus.

```
SQL> EXIT
```

4. Change to the directory where the text version of the DDL log file resides.

```
$ cd /u01/app/oracle/diag/rdtms/orcl/orcl/log
```

```
> cd C:/app/diag/rdtms/orcl/orcl/log
```

5. List the contents of the log directory.

```
$ ls
ddl ddl_orcl.log debug debug.log hcs imdb test
$
```

6. View the ddl_orclcdb.log (ddl_orcl.log) file by using the cat command.

```
$ cat ddl_orcl.log
diag_ddl: CREATE TABLE TEST (name varchar2(15))
…
diag_ddl: DROP TABLE TEST
$
```

7. Change to the ddl directory and list the contents. The XML version of the DDL log file (log.xml) is located here.

```
$ cd ddl
$ ls
log.xml
$
```

8. Close the terminal window.